

Table of Contents

Click on any heading below to be taken directly to that section

<i>Introduction.....</i>	<i>3</i>
<i>Defining Scope.....</i>	<i>3</i>
Must Have	4
Should Have	5
Could Have	5
Won't Have	6
<i>Programming Paradigm</i>	<i>6</i>
<i>Application Architecture.....</i>	<i>9</i>
<i>Project Management Methodologies</i>	<i>11</i>
Agile Methodology	11
Waterfall Methodology	13
<i>Task Management Methodologies</i>	<i>14</i>
Kanban	14
Scrum	15
<i>Methodology Selection for this Project.....</i>	<i>16</i>
Project Management Approach	16
Task Management Approach	17
Project Planning Tools.....	18
<i>Client / Server Architecture Explained.....</i>	<i>19</i>
Components and Characteristics of Client/Server Architecture.....	20
Client.....	20
Server	21
Network	22
Protocol	23
Middleware	24
Application Logic	24
How does it work?	25
<i>How The Village uses Client / Server Architecture.....</i>	<i>26</i>
Overview	26
Client / Server Communication	27

Data Distribution	28
Data Security.....	28
Feature Distribution.....	29
Authorisation	30
Validation	30
<i>Entity Relationship Diagram.....</i>	<i>31</i>
<i>User Stories</i>	<i>32</i>
User Persona: Priya Sharma	32
User Persona: Marc Nguyen.....	33
User Persona: Destiny Johnson.....	34
User Persona: Selwyn Hanson	35
<i>Ethical Web Development Principles</i>	<i>36</i>
Web applications should work for everyone	37
Web applications should work everywhere	38
Web applications should respect a user’s privacy and security	39
Web developers should be considerate of their peers.....	40
<i>Wireframes</i>	<i>41</i>
<i>References</i>	<i>44</i>

Introduction

The Village is a web-based health and wellbeing tracking application designed to help users monitor changes in their mood and physical pain over time. The application allows users to record structured entries using rating scales, select predefined labels that describe their emotional or physical state, and optionally add contextual notes. These entries are then visualised through time-based graphs, enabling users to identify patterns or trends in their wellbeing.

The application also allows users to add event markers to their timeline, such as starting a new medication, beginning therapy, or experiencing a significant life event. These markers provide additional context that may help users understand potential factors influencing their mood or pain levels. By combining structured tracking with visualisation tools, The Village aims to support self-reflection and provide users with useful insights that may assist discussions with healthcare professionals.

The following document outlines the design and planning process for the application, including the software development methodologies used, system architecture considerations, data modelling decisions, and user-centred design elements such as personas and user stories.

Defining Scope

To ensure realistic delivery, features have been categorised using the MoSCoW prioritisation framework: Must Have, Should Have, Could Have, and Won't Have.

The Must Have items constitute the Minimum Viable Product (MVP) and define the baseline functionality required for initial release. Features categorised as Should Have and Could Have are treated as stretch goals and may be implemented in subsequent iterations if time permits.

Must Have

1. User account management

User accounts are essential for data ownership, privacy, and personalised tracking over time.

- Ability to create an account.
- Secure authentication (registration and login).
- Each user can only access their own data.

2. Theme selection

This supports accessibility and user comfort without significantly increasing technical complexity.

- Ability to toggle between predefined themes (e.g., light mode and dark mode).

3. Mood logging

Structured rating scales allow trend visualisation and analysis, while optional notes support qualitative self-reflection.

- Rate mood on a scale of 0–10.
- Select a mood from a predefined list.
- Optional free-text notes field.
- Date/time of entry (**occurred_at**, defaulting to current datetime if not specified).

4. Pain logging

Separating mood and pain entries allows independent tracking while enabling comparison across time.

- Rate pain on a scale of 0–10.
- Select affected body area from a predefined list.
- Optional free-text notes field.
- Date/time of entry (**occurred_at**, defaulting to current datetime if not specified).

5. Data visualisation

Visualisation is central to the application's value proposition, enabling users to identify patterns and correlations.

- Display mood and pain ratings on a time-based graph.
- X-axis: date.
- Y-axis: rating scale (0–10).
- Ability to view trends over selectable time ranges.

6. Event markers

Event markers provide contextual insight, allowing users to correlate trends with real-world changes.

- Ability to add dated events (e.g., medication changes, therapy commencement, significant life events).
- Events displayed as markers on the graph.

7. Resources page

As the application deals with sensitive wellbeing data, providing support resources demonstrates ethical consideration and responsible design.

- A static resources page containing mental health support information (e.g., Lifeline and other relevant community resources).

Should Have

8. Data export

Export functionality would support sharing data with healthcare professionals. However, it introduces additional complexity in formatting, file generation, and security.

- Ability to export graph data and/or entries in a shareable format (e.g., PDF or CSV).

Could Have

9. "Dopamine Button"

This feature adds positive engagement and novelty but does not contribute to the core tracking and analysis objectives of the application.

- A button that displays a random affirmation or motivational quote.

Won't Have

10. Social features

These features significantly increase architectural complexity, moderation requirements, and privacy risks.

- Friend connections.
- Messaging between users.
- Content sharing.

11. Habit tracking

While valuable, habit tracking expands the data model and user interface complexity.

- Medication adherence tracking.
- Water intake logging.
- Sleep tracking.

12. Calendar integration and reminders

Notification systems and third-party integrations introduce additional backend complexity.

- Push notifications for things like taking medication and script refill reminders.
- Calendar syncing.

Programming Paradigm

The Village app will use a hybrid programming paradigm of functional and data-oriented with potential limited object-oriented principles.

The primary paradigm will be functional and will be used for the business logic including data transformations, Express middleware chains and React functional components with hooks.

An example React component may look like the following:

```
import { useState } from 'react';

function MoodLogger() {
  const [mood, setMood] = useState('');

  const logMood = () => fetch('/api/moods', {
    method: 'POST',
    body: JSON.stringify({ mood })
  });

  return (
    <input value={mood} onChange={e setMood(e.target.value)} />
    <button onClick={logMood}>Log Mood</button>
  );
}
```

This is correct for our MVP as it has lower complexity with easier unit testing. It's faster to develop with reduced architectural overhead and minimal boilerplate. The functional paradigm keeps logic deterministic and predictable which is critical for mental health, pain and event tracking systems where calculations must be reliable.

Secondary to this, we will use data-oriented design for MongoDB schema modelling and document structuring around usage patterns. As MongoDB is document-based, it is not relational. Therefore, data is structured by access pattern (e.g.: mood logs and pain logs embedded by the user), queries drive the schema design and aggregations support analytics. For our MVP, data modelling will prioritise simplicity and retrieval efficiency.

```
// models/MoodEntry.js
const mongoose = require('mongoose');

const MoodEntrySchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  value: { type: Number, required: true },
  createdAt: { type: Date, default: Date.now }
});

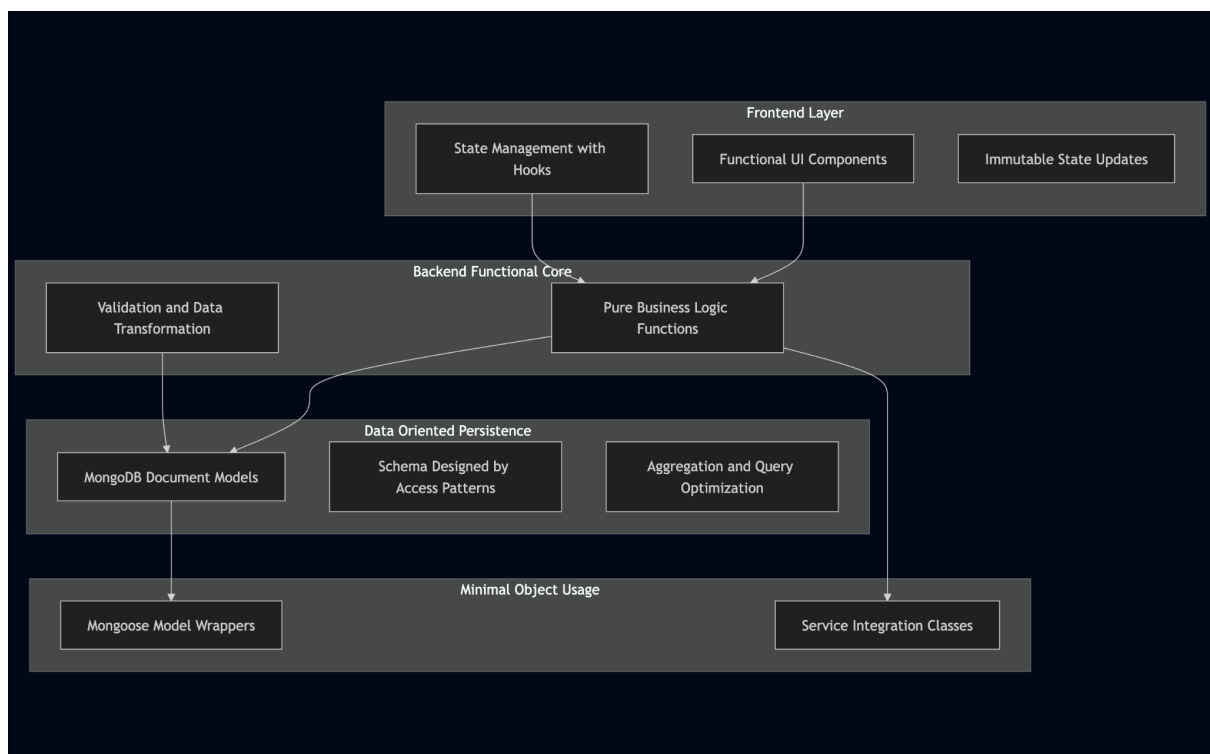
module.exports = mongoose.model('MoodEntry', MoodEntrySchema);
```

Lastly, we will have limited object-oriented usage and it's patterns will only appear in mongoose models for schema abstraction, and possibly service wrappers down the road of development. There will be no deep inheritance or domain entity class hierarchies. Our MVP will avoid heavy use of OOP as it increases complexity early on with little benefit. As we have floated the idea of email notifications, an example of this in OOP could look like this:

```
// services/EmailService.js
class EmailService {
  constructor(smtpConfig) { this.config = smtpConfig; }
  sendEmail(to, subject, body) { /* some logic here */ }
}
module.exports = EmailService;
```

In short, our system uses a functional-first approach for business logic and front-end components, data-oriented principles for MongoDB schema design and minimal OOP abstractions limited to schema definitions and potential service boundaries.

An example diagram of how this paradigm works together may look like this:



Application Architecture

The Village application will opt for a monolithic model-view-controller architecture. In comparison, microservices would introduce network overhead, service discovery complexity, deployment orchestration and increased DevOps requirements. As such, an MVP does not justify this complexity.

Our system architecture model will apply Mongoose schemas (User, MoodLog, PainLog, EventLog) with database validation rules and data access methods. This layer will handle persistence only. The model layer might look like this for example:

```
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true,
    unique: false,
  }
});

const UserModel = mongoose.model("User", UserSchema);

module.exports = {
  UserSchema,
  UserModel
}
```

Next, our View will render user interface and handle interaction with a React front end, functional components, state management via hooks and API consumption.

```
<MoodLogger /> // as shown above
```

Controllers must remain thin to coordinate. Ours will handle Express routes, request validation, calls to service/ business logic and JSON response formatting. For example:

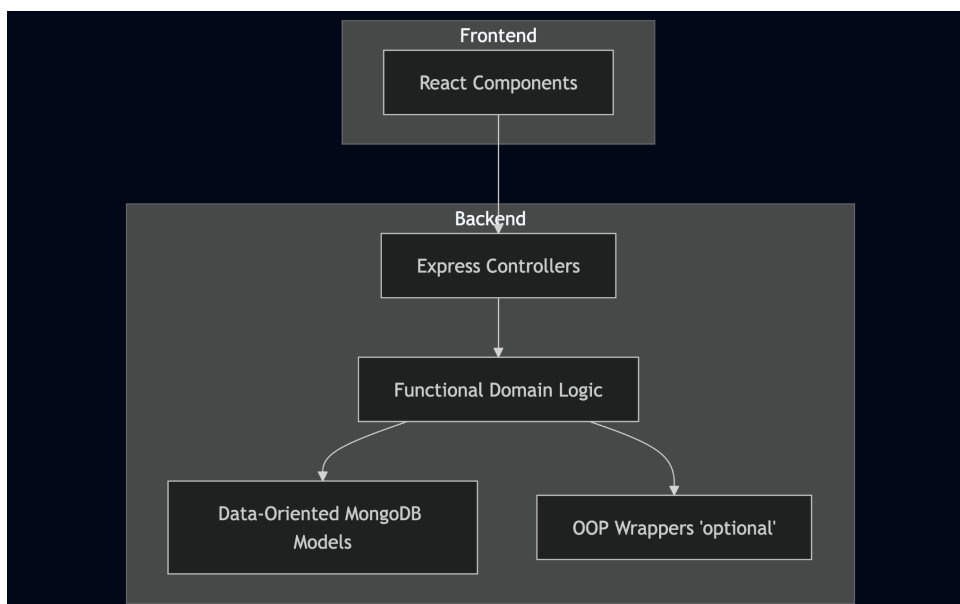
```
// controllers/pain.controller.js
const PainEntry = require('../models/PainEntry');
const { calculatePainScore } = require('../domain/pain.logic');

async function logPain(request, response) {
  const entry = await PainEntry.create(request.body);
  const score = calculatePainScore(await PainEntry.find({ userId:
request.body.userId }));
  response.json({ entry, score });
}
module.exports = { logPain };
```

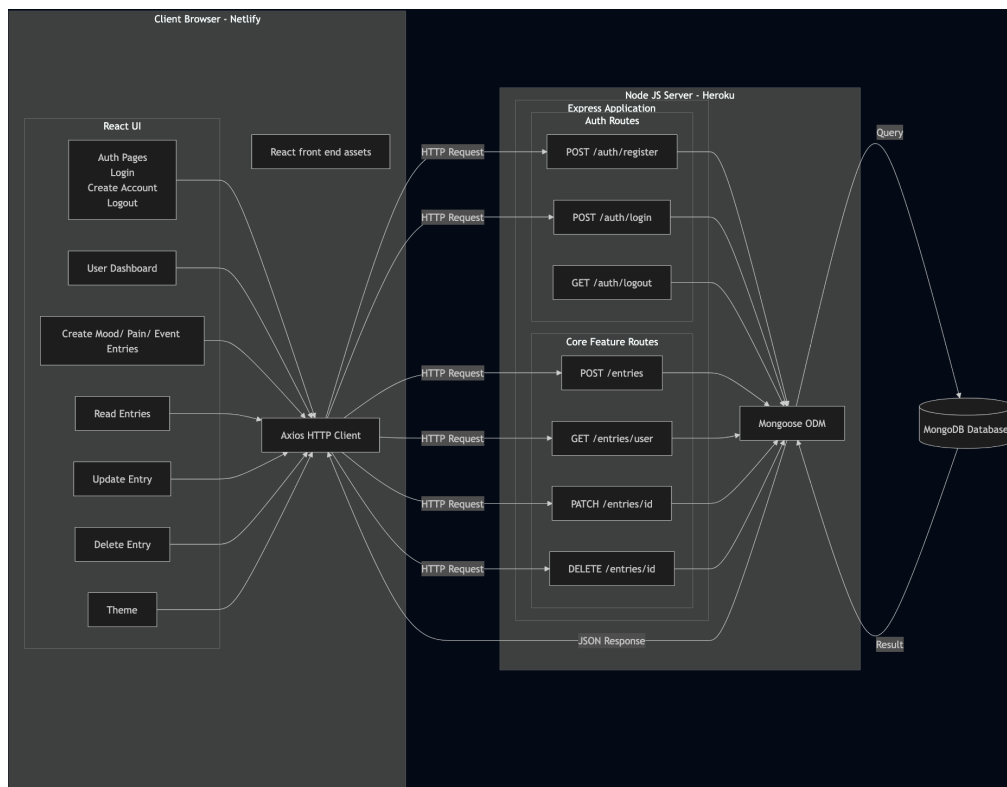
MVC is the correct option for our MVP as its core features are user authentication, mood, pain and event tracking. As such, we need clear separation of concerns, a predictable request flow, fast iteration and maintainability for scaling. MVC provides logical organisation and is simpler for onboarding team members. It is also the industry-standard pattern for MVP SaaS applications. Here is simple overview of the system:



The layered hybrid paradigm in MVC:



And an example of the full application architecture:



Project Management Methodologies

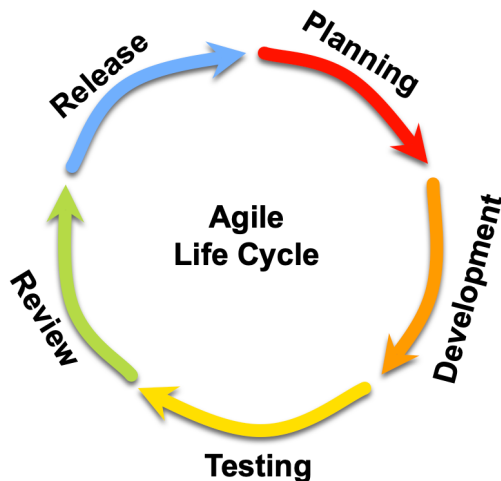
Agile Methodology

Agile is a project management approach used in software development that focuses on flexibility, collaboration, and incremental progress. Rather than attempting to define every requirement at the beginning of a project, Agile encourages teams to deliver small pieces of working functionality and continuously refine the product as new feedback becomes available. This allows development teams to adapt to changing requirements and improve the product throughout the development lifecycle (Beck et al., 2001).

In Agile development, work is typically organised into repeated cycles of planning, development, testing, and review. These cycles allow teams to evaluate progress regularly and adjust priorities where necessary. Agile approaches also emphasise communication between team members and encourage close collaboration when making design and implementation decisions.

Agile has become widely adopted in modern software development because it enables teams to respond to uncertainty and evolving requirements more effectively than traditional sequential approaches (Agile Alliance, n.d.).

Example Agile lifecycle



1. Planning
During the planning stage, the team reviews project priorities and determines which features or tasks should be addressed next. Requirements are clarified, tasks are broken down into smaller units of work, and responsibilities are assigned.
2. Development
In the development stage, team members implement the planned functionality by writing code, creating user interface components, and integrating system features according to the project requirements.
3. Testing
Testing involves verifying that the implemented functionality behaves as expected and does not introduce errors. This may include manual testing, automated testing, and checking that different parts of the system work together correctly.
4. Review
During the review stage, the completed work is evaluated to ensure it meets the intended requirements and quality standards. This may involve peer code review, validation of the implemented feature, and identifying improvements before release.

5. Release

The release stage involves making the completed functionality available within the application or preparing it for deployment. Once released, feedback can be gathered and used to inform the next planning phase.

6. (Repeat)

Because Agile development is iterative, this cycle repeats throughout the project, allowing the team to continuously refine the system and respond to feedback or changing requirements.

This iterative process supports continuous learning and gradual improvement of the system being developed.

Waterfall Methodology

Waterfall is a traditional project management methodology that follows a sequential and structured process. In a Waterfall model, each stage of the project is completed before the next stage begins. The typical phases include requirements gathering, system design, implementation, testing, and deployment (Atlassian, n.d.).

In this model, the majority of planning and design work occurs at the beginning of the project. Once development begins, changes to earlier decisions can be difficult or costly to implement because later stages depend on the completion of earlier ones.

Example Waterfall workflow



1. Requirements

During the requirements stage, the goals and functional requirements of the system are identified and documented. Stakeholder needs are analysed to determine what the system must achieve before development begins.

2. Design

In the design stage, the system architecture and technical structure are planned based on the documented requirements. This may include designing the database structure, application architecture, and user interface layout.

3. Development

During development, the planned system components are implemented by writing the necessary code and building the application according to the design specifications.

4. Testing

Testing is conducted to verify that the system functions correctly and meets the original requirements. This stage involves identifying and resolving bugs, as well as ensuring the system behaves as expected.

5. Deployment

In the deployment stage, the completed system is released for use. This may involve installing the software on a production environment or making the application available to end users.

Waterfall can be effective for projects where requirements are well defined and unlikely to change. However, it may be less suitable for projects where requirements evolve during development or where experimentation and iteration are beneficial (Atlassian, n.d.).

Task Management Methodologies

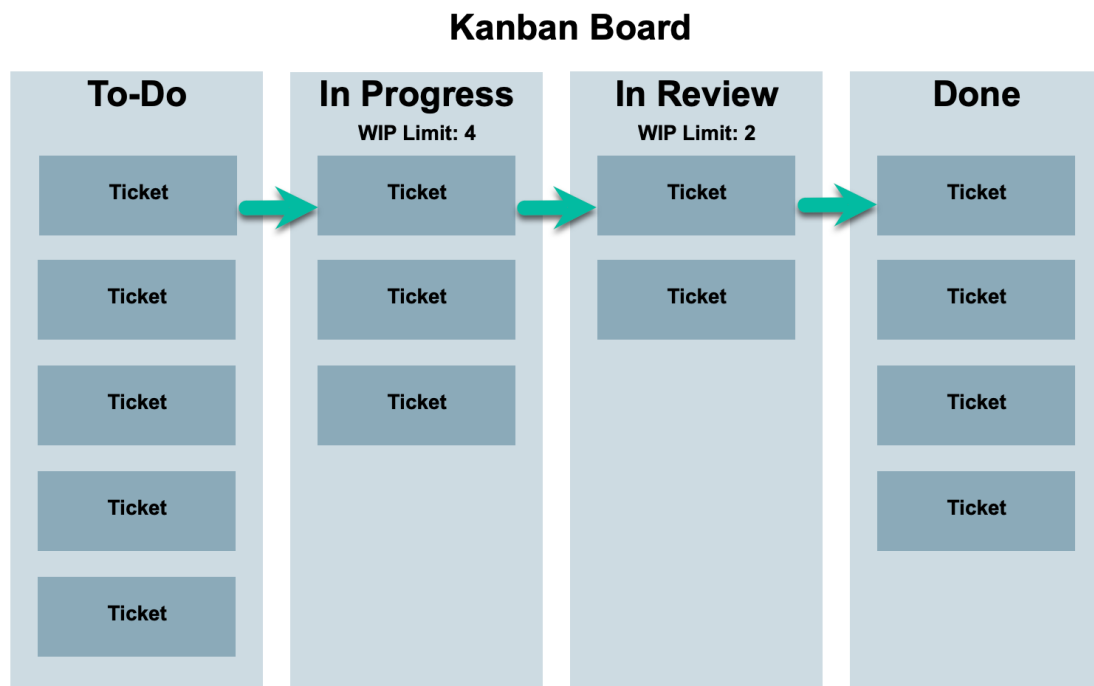
Kanban

Kanban is a workflow management method used to visualise and manage tasks as they move through different stages of completion. Work items are typically represented as cards on a board, which move across columns representing stages such as backlog, in progress, and completed.

The main objective of Kanban is to improve the flow of work by making tasks visible and limiting the amount of work being performed at any one time. These limits, known as

Work-In-Progress (WIP) limits, help teams avoid overloading themselves and encourage completion of tasks before new ones are started (Anderson and Carmichael, 2016).

Example Kanban workflow



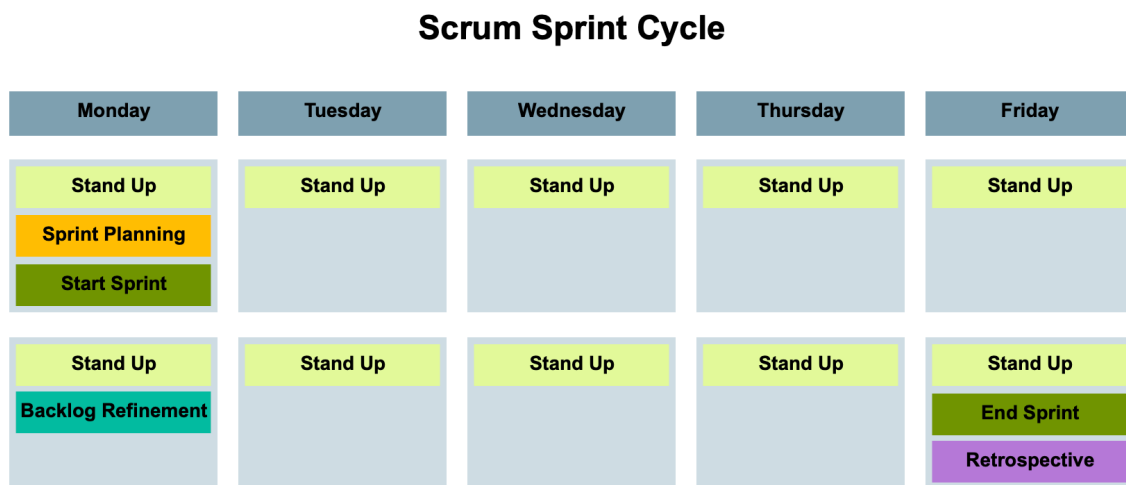
Each task is represented as a ticket that moves across workflow stages, beginning in the “To Do” column and progressing through stages such as “In Progress”, “In Review”, and “Done”. Work-in-progress (WIP) limits are applied to certain stages to reduce bottlenecks and ensure that team members focus on completing tasks before starting new ones.

Kanban supports continuous delivery because tasks can be completed and released individually rather than waiting for a fixed development cycle.

Scrum

Scrum is another Agile framework that organises work into structured development cycles called sprints. Each sprint typically lasts between one and four weeks and includes several defined activities such as sprint planning, daily meetings, sprint review, and retrospective (Schwaber and Sutherland, 2020).

Example Scrum sprint cycle



At the beginning of the sprint, the team conducts Sprint Planning to select tasks from the product backlog and define the work that will be completed during the sprint. Daily stand-up meetings are held throughout the sprint to allow team members to briefly share progress, identify obstacles, and coordinate work. During the sprint, activities such as backlog refinement may also occur to clarify and prioritise upcoming tasks. At the end of the sprint, the team completes the work cycle with a Sprint Review and Retrospective, where the completed work is evaluated and the team reflects on ways to improve the development process for the next sprint.

Scrum introduces defined roles, including the Product Owner, Scrum Master, and Development Team, to support communication and coordination during the development process.

Methodology Selection for this Project

Project Management Approach

For the development of The Village, the team has chosen to adopt an Agile project management approach rather than a Waterfall model.

Agile is well suited to this project because the application is being developed in stages across multiple assessments, including planning, frontend development, backend implementation, and deployment. Using an Agile approach allows the team to progressively build the system while refining features and priorities based on ongoing discussions and feedback.

Additionally, all team members are balancing this project alongside other commitments. The flexibility offered by Agile enables the team to adapt planning and task allocation based on availability while still maintaining steady progress.

Task Management Approach

To manage day-to-day development tasks, the team is using Kanban through a Jira board.

Kanban was selected instead of Scrum because it allows tasks to move continuously through the workflow without requiring strict time-boxed sprints. This approach provides greater flexibility for a team working on this project part-time and enables members to begin tasks when they have capacity.

The Jira Kanban board is used to visualise the workflow and track progress across several stages, including “To Do”, “In Progress”, “Review”, and “Done”.

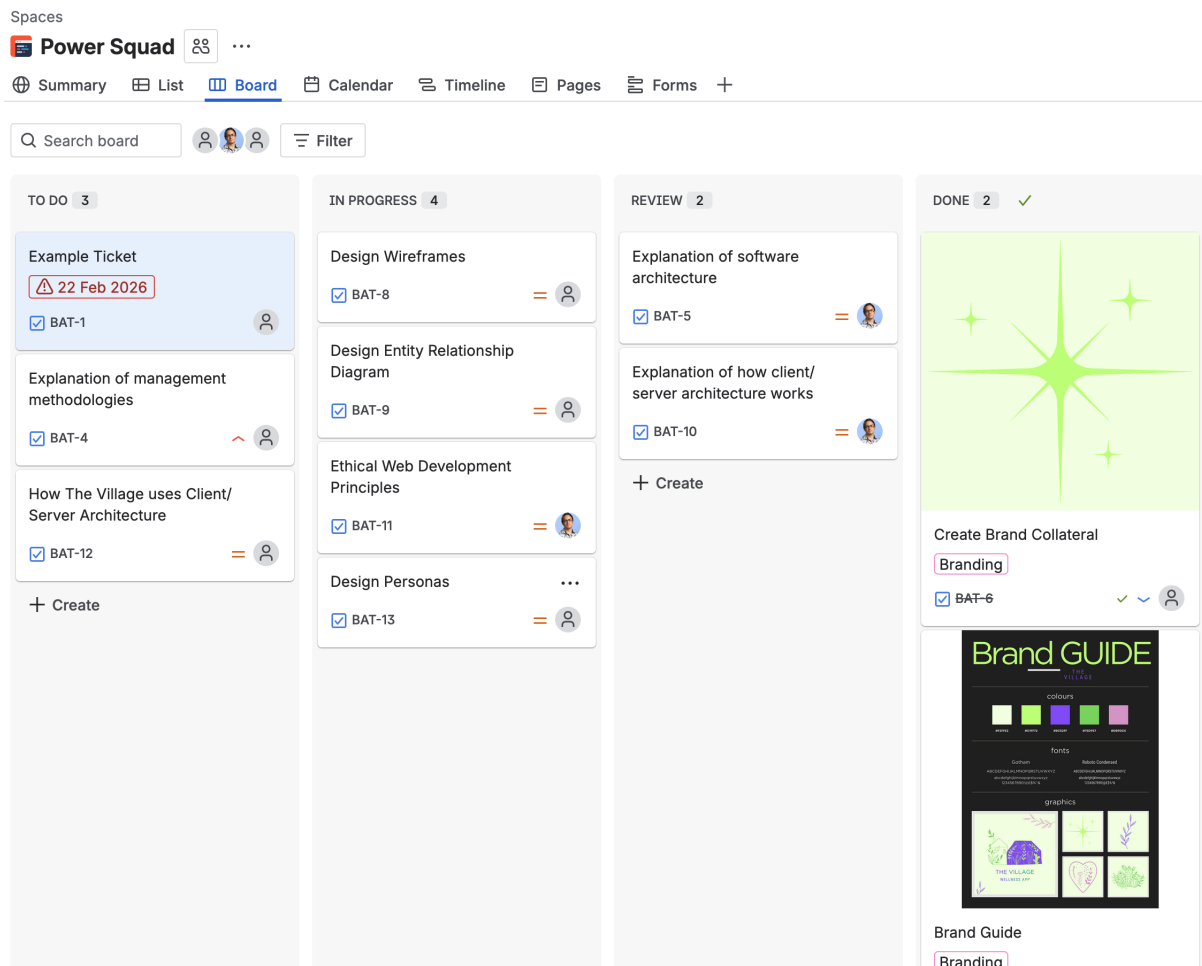
Each task is represented as a Jira ticket and moves across these stages as work progresses. This allows the team to clearly see the current status of tasks and identify any bottlenecks.

Project Planning Tools

Two main tools are used to support collaboration and planning within the project:

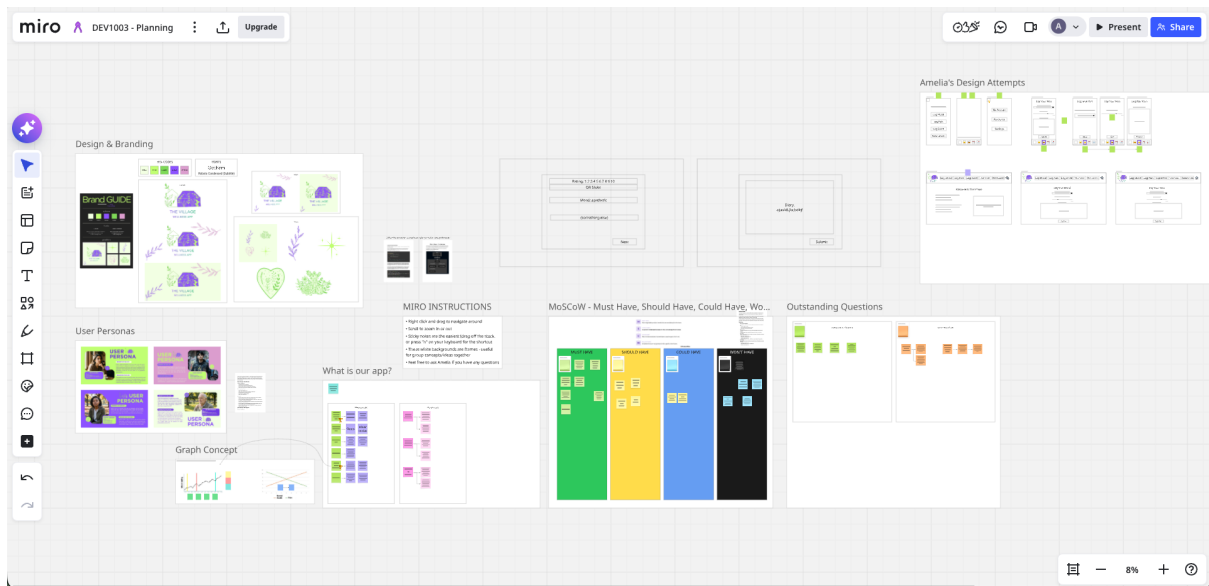
Jira

Jira is used to manage development tasks and organise the workflow using a Kanban board. Tasks are recorded as tickets and assigned to team members, allowing progress to be tracked transparently.



Miro

Miro is used for collaborative planning activities such as brainstorming features, mapping system components, and organising higher-level project ideas before they are converted into actionable Jira tasks.

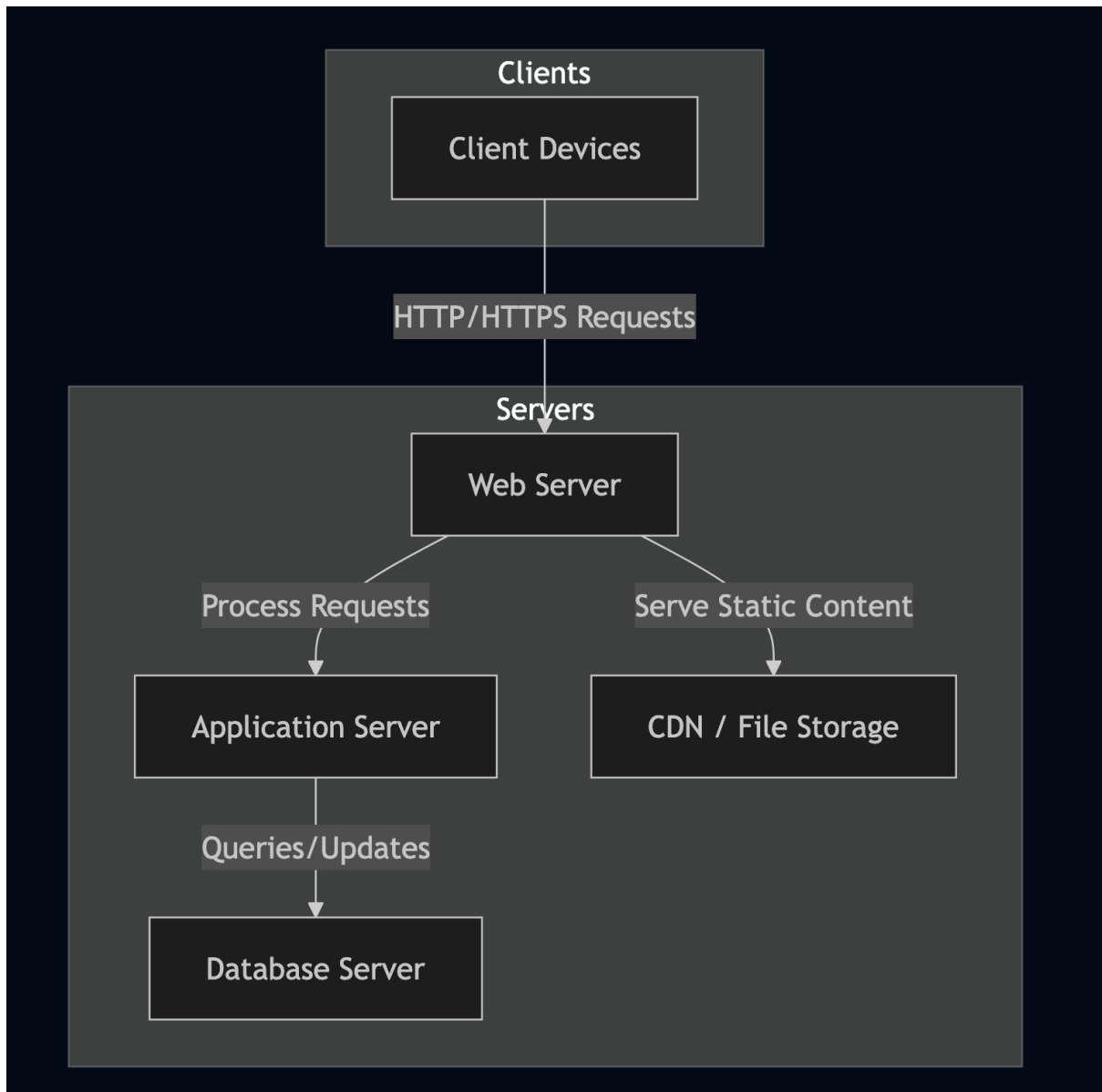


Client / Server Architecture Explained

When browsing the web, sending an email, streaming Netflix or playing a web-based game, you're interacting with a network-based computing system called Client/ Server architecture which is a method of organising and distributing resources between clients and servers.

The architecture is a model that allows communication and data exchange between applications over a single or multiple server points.

The model is divided into two parts:



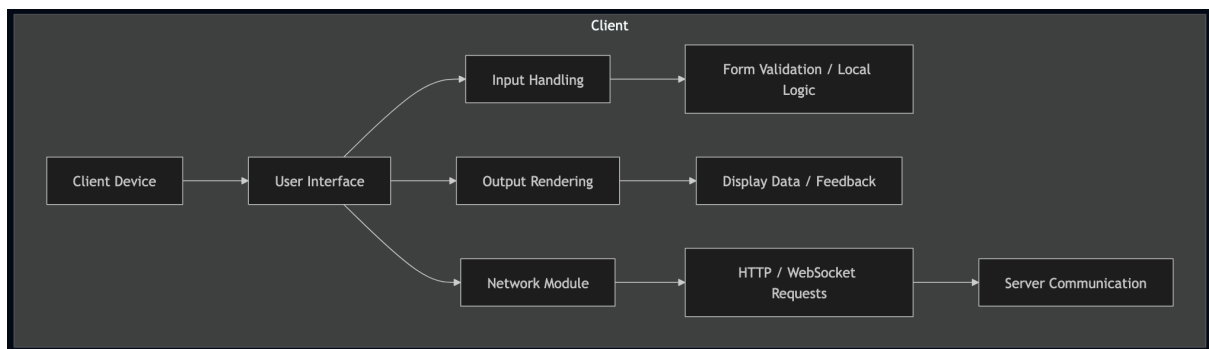
Components and Characteristics of Client/Server Architecture

Client

The client is an application that requests data and/ or services from the server such as storage, calculations and other functions.

A client device is consumer-facing and generally includes web browsers, desktop apps or mobile apps that users interact with. They communicate with the server to delegate tasks, make transactions or retrieve information.

Client and server ends can be different in requirements with hardware and software. They may also come from different vendors. Though, both interact directly with a transport layer protocol to establish communication for sending and receiving data.



Server

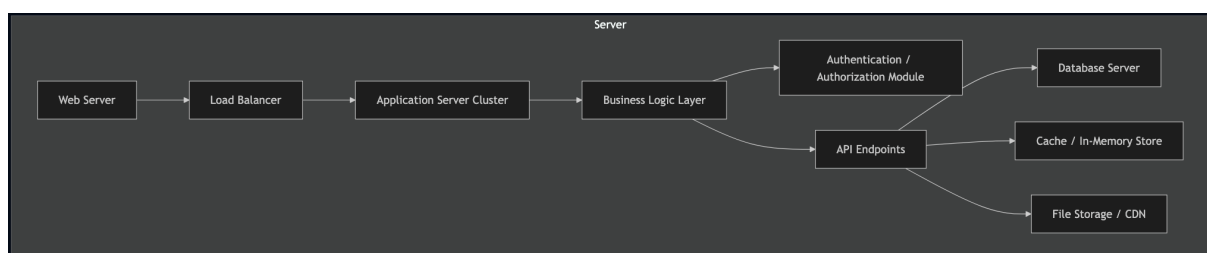
The server is an application that processes said client requests and sends responses or performs actions based on those requests. The server and client can both live on the same computer or different devices on the network.

A server offers services or solutions to clients over the network. They can handle processing client requests, which can include tasks such as file storage, database access and application hosting, as well as back-end activities such as computations, business logic and data management which reduces what clients need to handle.

A single server can respond with multiple services simultaneously, but each service must run its own server program.

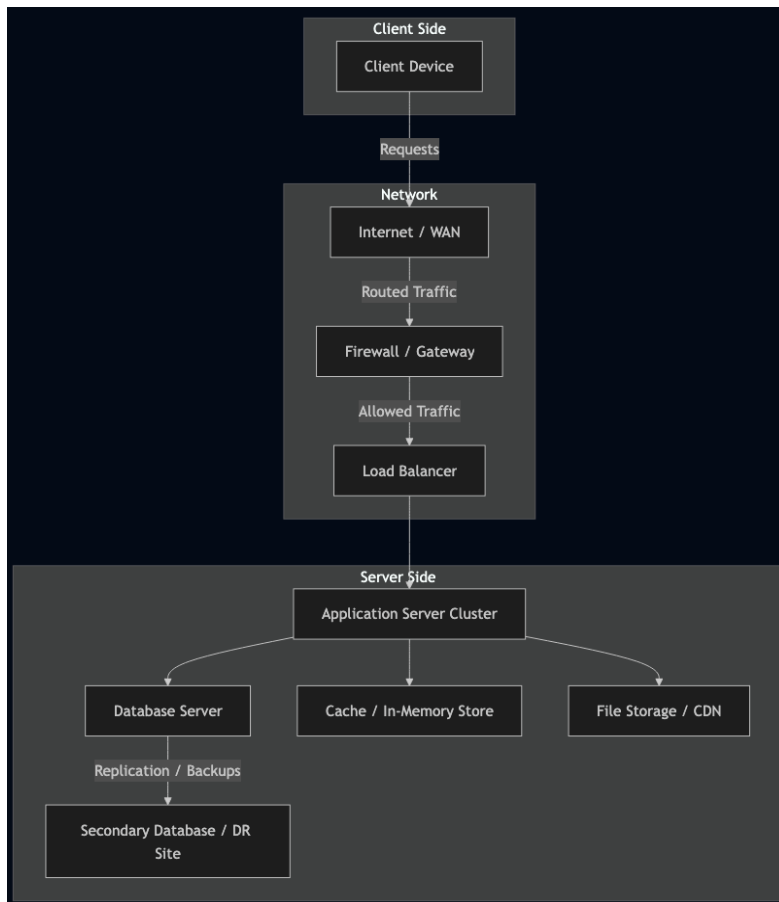
Some examples of servers are:

- **File Servers:** Upon saving documents on services like Microsoft Office, you are interacting with file servers. This type of server stores data centrally and allow multiple clients to use it.
- **Email Servers:** Emails are the primary communication method of businesses due to convenience and speed. Multiple server components work in tandem to deliver emails between users across different mail servers.
- **Web Servers:** Very performant and powerful servers that host websites. Web clients can access these servers through DNS or IP addresses.



Network

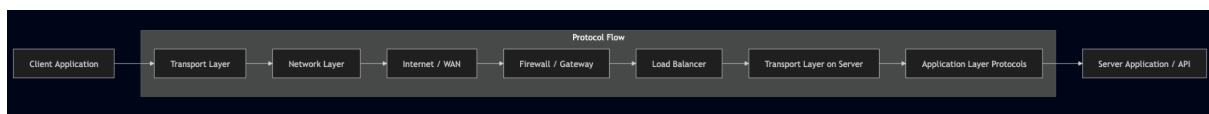
This serves as the channel through which clients and servers are connected for data transfer between them. These can range from local area networks within a single building, to wide area networks and the internet. It acts as the intermediary, facilitating the interchange of requests and responses between the clients and servers, which influences the speed and reliability of these interactions.



Protocol

These are rules that define how data is exchanged between the clients and servers. They ensure the communication is secure, ordered and understandable. Common protocols are HTTP or HTTPS for web services, FTP for file transfers and SMTP for email. They assist in bridging communication between different systems, independent of their technology stack.

Both client and server computers require a full stack of protocols where the transport protocol uses lower layer protocols to send and receive individual messages.



Middleware

Middleware is a software layer within a server application that intercepts and processes requests and responses during communication between the client and the server. It allows developers to insert reusable logic into the request-response pipeline, enabling tasks such as authentication, logging, data transformation, and error handling before the request reaches the core application logic.

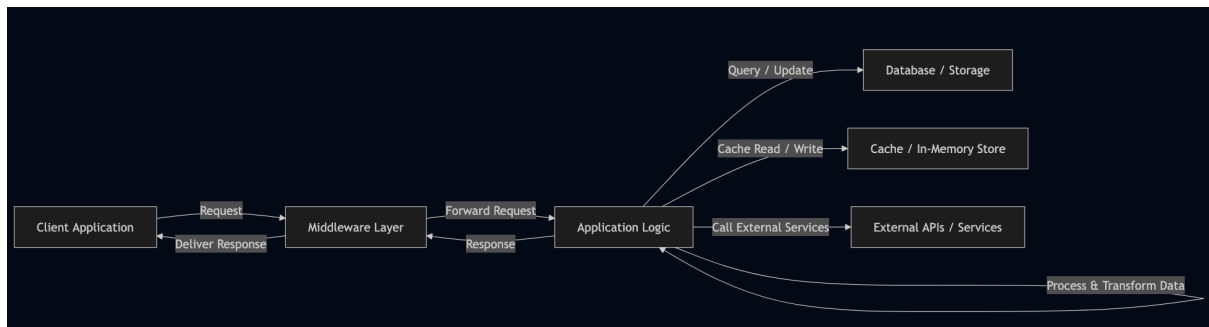
Additionally, middleware can enforce authorization by ensuring that only clients with appropriate permissions can access certain resources or perform specific operations. It can also validate incoming requests, confirming that data conforms to expected formats, types, and business rules before it is processed. This helps reduce errors, improve security, and maintain the integrity of the application.



Application Logic

This is the processes and code that determine how the server responds to requests involving big data processing, business rules and workflows on the server side. It makes sure the server correctly interprets client requests, performs data manipulations or necessary calculations and delivers appropriate responses.

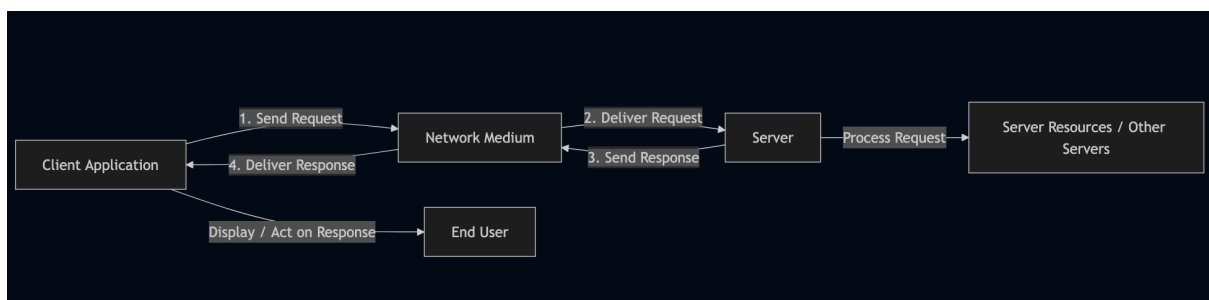
By centralising decision-making and processing, the application logic ensures consistent behaviour, enforces security and compliance rules, and prepares data in a format that clients can directly consume, enabling reliable and scalable server-side operations.



How does it work?

In four basic steps, the client/server architecture works like so:

1. The client sends a request to the server using the network medium. The request can be a query, command or message.
2. The server receives the request and processes it according to its data and logic. The server may access its own resources or other servers to fulfil the request.
3. The server then sends a response back to the client using the network medium. The response can be an acknowledgement, data or an error message such as a 500 HTTP error.
4. The client receives the response and displays it to the user or performs further actions based on it.



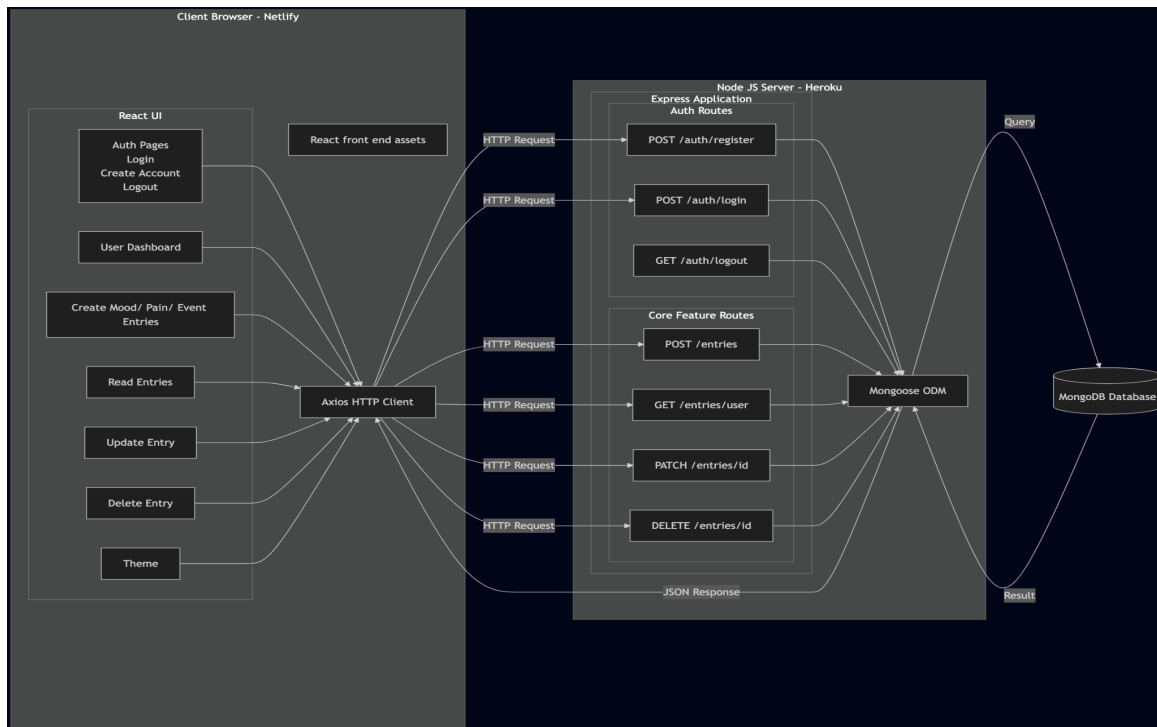
How The Village uses Client / Server Architecture

Overview

The Village is built using a client/server architecture typical of the MERN stack. The client side consists of a React single page application (SPA) hosted on Netlify, which runs in the user's web browser. This frontend is responsible for rendering the user interface, managing client-side interactions, and sending HTTP requests to the backend when data needs to be created, retrieved, or updated.

The server-side is a Node.js application hosted on Heroku and built using the Express framework. The Express server handles incoming requests from the React client, processes application logic, and exposes RESTful API endpoints. Routes are organised into groups, with authentication-related routes handled separately from core feature routes to keep the backend structure modular and maintainable.

Communication between the client and server occurs through HTTP requests sent using the Axios HTTP client in the React application. The server then interacts with the database through Mongoose, an Object Data Modelling (ODM) library that provides a structured way to define schemas and manage data in MongoDB. This layered architecture separates the user interface, application logic, and data storage, making the system easier to develop, maintain, and scale.



Client / Server Communication

The React client (hosted on Netlify) communicates with the Express API (hosted on Heroku) over HTTPS using Axios. The API exposes RESTful endpoints grouped by purpose (e.g. `/api/v1/auth/*` for authentication, `/api/v1/entries/*` for mood / pain / event CRUD), and the client uses the standard HTTP verbs - **POST** for create, **GET** for read, **PATCH** for updates, and **DELETE** for removals. We will keep a single Axios instance configured from an environment variable (e.g. `REACT_APP_API_URL`) so builds point to the correct Heroku URL; the server will enable CORS for the Netlify origin and use clear status codes and JSON payloads (**Content-Type: application/json**) with a simple response envelope like `{ success, data, error }`.

On our client, we'll create an Axios instance with sensible defaults (baseUrl, timeout) and interceptors to attach auth tokens to **Authorization** headers and to handle common error cases centrally. Implement standard retry/backoff or user-facing error messages for network failures and will consider optimistic UI patterns for quick **PATCH** / **DELETE** interactions with rollback on server error. On the server, validate and sanitise incoming JSON, use Mongoose schemas to shape responses, and never return sensitive fields (e.g. hashed passwords). Finally, we will wire deployment and runtime settings through environment variables (API URL, JWT secret, DB URI) so local, staging and production environments remain consistent and secure.

Data Distribution

We will treat the Express API / MongoDB as the single source of truth: the server owns persistent data and the client requests exactly what it needs via JSON REST endpoints (e.g. **GET /api/v1/entries?type=mood&limit=20&page=2**). Our app may implement server-side pagination (if necessary), filtering and field projection (Mongoose **limit()**, **skip()**, and **select()** / **.lean()**) so responses are compact and predictable, and use MongoDB indexes on common query fields (**userId**, **createdAt**, entry type) and aggregate pipelines for summary queries (weekly mood averages) to keep response times low. On the server, we'll map database documents to clear response DTOs (data transfer objects) that exclude sensitive fields (never sending hashed passwords or internal tokens).

On the client, we will be keeping a minimal local cache of fetched data (React state, Context or a small library like React Query) and prefer revalidation on focus or after mutations. For mutation workflows (create/ patch/ delete), send concise requests (IDs and changed fields) and use optimistic updates with rollback on error to keep the UI snappy. Caching critical read responses and queue writes in IndexedDB/localStorage while the service worker is offline, then sync them when connectivity returns, for offline resilience. Finally, we'll document and version the API (simple OpenAPI/markdown) so frontend and backend teams - or future markers - understand the shape and access patterns for each resource.

Data Security

The Village implements end-to-end protections: all client/server traffic uses HTTPS and TLS, and our API will enforce secure headers via Helmet and well-configured CORS for the Netlify origin. User credentials are hashed using Node's **cryptojs** on the server, and authentication uses short-lived JWTs with a refresh-token pattern; tokens are sent in **Authorization: Bearer <token>** headers on every request. Secrets and configuration (JWT secret, DB URI) are kept out of source control and managed with Heroku and Netlify environment variables; locally we'll access them via Node's **process.env** (set in the shell or via platform CLIs). We do not commit secrets in the repository.

On the backend we enforce input validation and sanitisation (Mongoose schema validation plus libraries like **express-validator**) to prevent injection and malformed data and apply rate limiting (e.g. **express-rate-limit**) to limit abuse. Data at rest is protected by the database provider (enable encryption at rest and least-privilege DB users in MongoDB Atlas), regular backups, and minimal exposure of sensitive fields in API responses. We log auth events and errors with concise, privacy-aware logs for incident review, and schedule periodic security checks and dependency updates to keep our stack resilient.

Feature Distribution

The Village splits responsibilities so the client (React on Netlify) handles all user-facing behaviour: rendering lists and forms for mood/ pain/ event entries, client-side validation and accessibility affordances, optimistic UI updates for PATCH/DELETE flows, and local theme toggling (dark/ light) with immediate UI persistence. The frontend uses Axios to call the API for full CRUD operations - **POST /api/v1/entries**, **GET /api/v1/entries**, **PATCH /api/v1/entries/:id**, **DELETE /api/v1/entries/:id** - and keeps a minimal local cache (React state/Context or a light cache library) that revalidates after mutations. Theme changes are applied instantly in the UI and persisted with a concise **PATCH /api/v1/users/:id/settings** request so settings survive across devices.

The server (Express on Heroku) owns business logic, persistence and access control: routes are organised into **auth** (login, logout, refresh) and **core** (entries, user settings) groups, backed by Mongoose models for **User** and **Entry** (type: mood|pain|event). The API enforces validation, ownership checks (only the entry owner can PATCH/DELETE), pagination and field projection for list endpoints, and returns DTOs that exclude sensitive fields. Authentication is checked via **Authorization: Bearer <token>** on every protected route; the server also handles **createdAt** / **updatedAt**, optional soft deletes for safety, and concise audit logging for important events. This separation keeps the client lightweight and responsive while ensuring the server remains the single source of truth for data integrity.

Authorisation

The Village implements token-based authorisation: the server issues short-lived JWT access tokens (signed with our JWT secret) and longer-lived refresh tokens tracked server-side for revocation/ rotation. On the backend we use the **jsonwebtoken** library and an Express **verifyToken** middleware that validates tokens, attaches **req.user** (id and scopes/role), and rejects invalid or expired tokens. Route-level guards (**requireRole('admin')**, **requireScope('entries:write')**) enforce role or scope checks, and ownership is enforced at the DB layer by always filtering updates/deletes by **userId** (e.g. **Entry.findOneAndUpdate({ _id, userId: req.user.id }, ...)**) rather than trusting client input. This keeps the principle of least privilege, supports simple RBAC (role-based access control), and makes token revocation (via a refresh-token store or blacklist) possible if credentials must be invalidated.

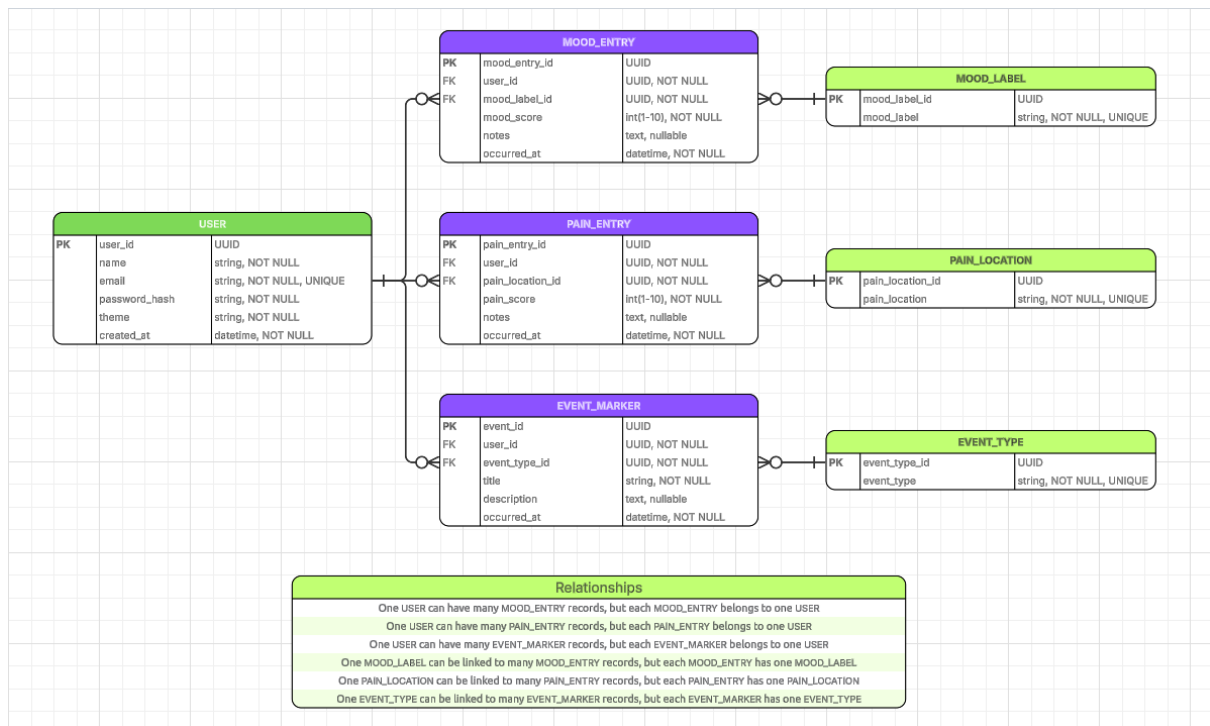
On the client we attach **Authorization: Bearer <token>** using an Axios instance and interceptors; a React Router private-route pattern prevents unauthenticated routes from rendering. The Axios interceptor handles 401s by calling **/api/v1/auth/refresh** to rotate tokens and retry the original request. For our project, we may persist refresh tokens in **localStorage** (with short access-token lifetimes) but we mitigate XSS (cross-site-scripting) risk through CSP (content security policy), input sanitisation and minimal use of **innerHTML**. We also add automated tests (Jest for client guards and supertest server middleware) and concise audit logs for login, token refresh and denied access to help detect and investigate abnormal authorisation events.

Validation

The Village implements server-side validation using **express-validator** middleware in our Express API so every incoming request is checked before reaching business logic or the database. Validation chains (e.g. **body()**, **param()**, **query()** rules) run as route middleware and results are collected with **validationResult()**; if checks fail the server responds with a consistent 400 payload describing the errors, preventing invalid or malicious data from progressing. This middleware approach keeps validation collocated with routes while remaining reusable across endpoints.

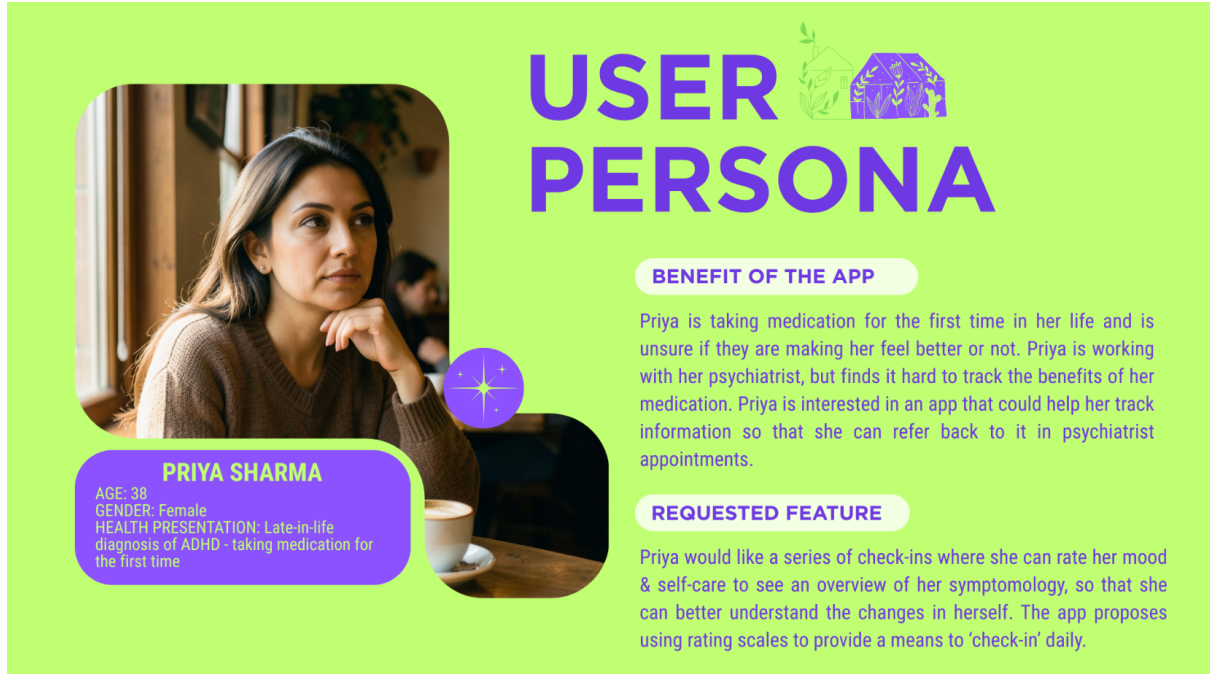
We centralise rules by exporting small validator modules per resource (e.g. **registerValidator**, **entryCreateValidator**) and prefer **checkSchema** or chained validators for complex inputs so rules are explicit and testable. Sanitisation helpers are applied where needed and Mongoose schema validation (**runValidators: true**) is used as a final safety net before writes. Test coverage (Jest and supertest) targets bad payloads and edge cases, and the server normalises errors into a simple **{ success: false, errors: [...] }** structure so the client can surface friendly messages and the backend remains auditable and robust.

Entity Relationship Diagram



User Stories

User Persona: Priya Sharma

A user persona card for Priya Sharma. It features a green background. On the left, there is a circular portrait of a woman with dark hair, wearing a brown sweater, resting her chin on her hand. Below the portrait is a purple rounded rectangle containing her name and details. To the right of the portrait is a purple starburst icon. Further right, the text 'USER PERSONA' is written in large, bold, purple letters, with a small house icon above the word 'PERSONA'. Below this, there are two white rounded rectangles with purple text: 'BENEFIT OF THE APP' and 'REQUESTED FEATURE'. The background of the card also features a faint illustration of a house with a plant.

USER PERSONA

PRIYA SHARMA
AGE: 38
GENDER: Female
HEALTH PRESENTATION: Late-in-life diagnosis of ADHD - taking medication for the first time

BENEFIT OF THE APP

Priya is taking medication for the first time in her life and is unsure if they are making her feel better or not. Priya is working with her psychiatrist, but finds it hard to track the benefits of her medication. Priya is interested in an app that could help her track information so that she can refer back to it in psychiatrist appointments.

REQUESTED FEATURE

Priya would like a series of check-ins where she can rate her mood & self-care to see an overview of her symptomology, so that she can better understand the changes in herself. The app proposes using rating scales to provide a means to 'check-in' daily.

Purpose for using the app:

- Track effectiveness of new medication.
- Monitor emotional changes during dose adjustments.
- Provide structured data to her psychiatrist during appointments.
- Identify whether improvements are gradual, stable, or inconsistent.

What is important to this user:

- Clear mood rating scales (0–10).
- Ability to select specific emotional states from a predefined list.
- Visual trend analysis over weeks and months.
- Event markers for medication changes.
- Ability to add contextual notes.

User stories:

- As Priya, I want to record my mood on a 0–10 scale each day so that I can determine whether my medication is improving my mental health over time.

- As Priya, I want to select a predefined mood label (e.g., anxious, calm, irritable) so that I can categorise my emotional state consistently.
- As Priya, I want to add optional notes to my mood entries so that I can provide context for my psychiatrist during appointments.
- As Priya, I want to add an event marker when I start or change medication so that I can visually correlate changes in mood with treatment adjustments.
- As Priya, I want to view my mood history on a graph so that I can identify trends and patterns over weeks or months.

User Persona: Marc Nguyen



USER PERSONA

BENEFIT OF THE APP

Marc has struggled to make friends for most of his school-life. Marc now suffers from Social Anxiety and finds that writing his feelings down during the day helps him to regulate and reflect. Marc also likes to try new social skills and then note how effective they were..

FEATURE REQUESTED

Marc needs an app where he can add to the same journal entry throughout the day. The app proposes to add a journal feature with timestamps - to support mood tracking and self-reflection.



MARC NGUYEN
AGE: 15
GENDER: Non-Binary
HEALTH PRESENTATION: Social Anxiety related to attending school without friendship connections

Purpose for using the app:

- Reflect on emotional responses to social interactions.
- Track anxiety patterns throughout the school day.
- Improve self-awareness and emotional regulation.

What is important to this user:

- Ability to log multiple entries in a day.
- Optional free-text notes for journalling.

- Simple, non-overwhelming interface.
- Clear visual feedback on mood fluctuations.

User stories:

- As Marc, I want to log mood entries throughout the day so that I can reflect on how specific social interactions affect my emotional state.
- As Marc, I want to include free-text reflections with timestamps so that I can track how my thoughts and feelings change over time.
- As Marc, I want to see my mood patterns visually so that I can better understand triggers related to school and social situations.
- As Marc, I want my entries to be private and accessible only to me so that I feel safe recording personal experiences.

User Persona: Destiny Johnson




USER PERSONA

BENEFIT OF THE APP

Destiny was in a car accident when she was a teenager, and she manages chronic pain daily. Destiny has found numerous apps that track mental health but not pain. Destiny would like to compare her pain on weeks where she works longer days at work versus weeks when she works less.

DESTINY JOHNSON

AGE: 21
GENDER: Female
HEALTH PRESENTATION:
Chronic pain after a car accident five years ago

REQUESTED FEATURE

Destiny wants a way to rate her pain and then compare these levels, so that she can see how her pain levels vary. The app proposes offering pain rating scales as well as mental health ratings, so that pain levels can be tracked.

Purpose for using the app:

- Monitor chronic pain intensity and location.
- Identify patterns related to work schedules or activity levels.
- Understand the relationship between physical pain and mood.

What is important to this user:

- Accurate pain rating scale (0–10).
- Structured body location selection.
- Ability to compare pain over weeks.
- Overlay of mood and pain on the same timeline.
- Event markers for treatment changes.

User stories:

- As Destiny, I want to record my pain level on a 0–10 scale so that I can monitor fluctuations in pain intensity.
- As Destiny, I want to select the affected body area from a predefined list so that I can consistently track where pain occurs.
- As Destiny, I want to compare my pain levels across different weeks so that I can identify patterns related to work schedules or physical activity.
- As Destiny, I want to view mood and pain ratings together on the same timeline so that I can understand how physical pain affects my emotional wellbeing.

User Persona: Selwyn Hanson

BENEFIT OF THE APP

Selwyn has experienced a rough twelve months. Selwyn is a victim/survivor of elder abuse perpetrated by his two grandchildren, who had been his 'carers' for the past 5 years. Selwyn has recently moved into aged care and has been experiencing symptoms of depression, managed by his GP.

REQUESTED FEATURE

Selwyn struggles to hold a pen but can type on a device. Selwyn's GP is wanting Selwyn to track his moods and symptoms over a 3-month period so that he can assess if counselling and socialising at the aged care facility are improving Selwyn's mental health and wellbeing. The app proposes to allow users to track their moods on a graph, which can be exported for health professionals, such as GPs.



SELWYN HANSON
AGE: 76
GENDER: Male
HEALTH PRESENTATION: Depression triggered by moving into aged care after Elder Abuse

USER PERSONA



Purpose for using the app:

- Track mood changes over a longer period (e.g., 3 months).
- Support discussions with his GP.
- Monitor the impact of counselling and increased social engagement on overall wellbeing.

What is important to this user:

- Simple and accessible interface.
- Clear visual representation of trends.
- Minimal complexity.
- Option to export or share data (future enhancement).

User stories:

- As Selwyn, I want a simple interface to log my mood so that I can easily track my wellbeing without writing by hand.
- As Selwyn, I want to track my mood over a multi-month period so that my GP can assess whether counselling and social engagement are improving my mental health.
- As Selwyn, I want to export my mood data so that I can share it with my GP during appointments.

Ethical Web Development Principles

This MERN project, The Village, will be developed with a commitment to ethical web development. We will design our application to be accessible, inclusive, and usable for all users through progressive enhancement, responsive design, and performance-conscious implementation. We prioritise privacy and security by enforcing HTTPS, safeguarding user data, respecting tracking preferences, and providing transparency around data usage and export.

We also uphold professional responsibility by maintaining well-documented, tested, and version-controlled code in Git/ GitHub, and by engaging respectfully with the wider development community. These principles guide our technical and architectural decisions throughout the project.

Web applications should work for everyone

Progressive enhancement means building the application so that its core functionality works first and then layering more advanced features on top for browsers and devices that can support them.

This means basic content and functionality (forms, navigation, data display) should work even if the JavaScript fails or loads slowly. React interactivity enhances the experience, but the user should not be completely blocked without it and the pages should render meaningful content quickly before advanced UI behaviours initialise. Accessibility and semantic HTML come first; styling and dynamic features are added afterward. And finally, features that rely on modern APIs should degrade gracefully rather than break.

The Village prioritises accessibility, which means designing the health and wellness app so it can be safely and comfortably used by as many people as possible, including those with disabilities, temporary impairments, or heightened emotional sensitivity.

In practice, this means using semantic HTML so screen readers can accurately interpret content and ensuring full keyboard navigation for users who cannot use a mouse. Providing gentle colour schemes and readable typography to reduce visual strain as well as writing clear, simple language to support users experiencing stress, anxiety, or cognitive overload, and avoiding flashing elements or overwhelming UI patterns that could negatively impact vulnerable users.

In The Village, developing inclusive forms means designing input experiences that are clear, supportive, and non-judgemental, particularly given the sensitivity of mental health and pain information. Forms should use plain, empathetic language, clearly explain why information is being collected, and avoid assumptions about identity, relationships, or lived experience.

Validation feedback must be constructive and easy to understand, guiding users without creating frustration or shame. Input fields should support assistive technologies, allow flexible responses where appropriate, and minimise cognitive load so that users can complete tasks comfortably, even during moments of stress or emotional vulnerability.

Web applications should work everywhere

In The Village, responsive design ensures the interface adapts seamlessly across mobile phones, tablets, and desktops. Users seeking mental health and pain support may access the platform from a variety of environments, so layouts, typography, and interactions must remain clear, stable, and usable regardless of screen size or device capability. Touch targets, spacing, and text scaling are carefully considered to maintain comfort and readability. This approach ensures that the experience feels consistent and supportive, whether accessed privately at home or on the move.

Performance is prioritised to reduce load times and friction, particularly for users on slower or unstable connections. Efficient rendering, React lazy loading, optimised assets, and thoughtful data handling help ensure that users can access support quickly, without unnecessary technical barriers. In a wellbeing context, delays and unresponsive interfaces can increase frustration or distress, so speed and reliability directly contribute to user trust. Monitoring and optimisation are treated as ongoing responsibilities rather than one-time tasks.

Where appropriate, offline-first functionality with service-workers allows essential features to remain available during temporary connectivity loss. This improves reliability and ensures users can still access important information or continue interactions even when internet access is interrupted. Caching critical resources and preserving in-progress data reduces the risk of losing meaningful input. This resilience is especially important when users are engaging with personal reflections or sensitive material.

Clear, permanent, human-readable URLs enable users to bookmark, revisit, and share specific resources with confidence. Deep linking with React Router also supports accessibility and transparency by ensuring content can be directly accessed without

relying solely on client-side navigation. Structured routing improves search visibility and makes the platform easier to understand conceptually. It also allows support resources to be referenced reliably in external communications, strengthening continuity of care and access.

Web applications should respect a user's privacy and security

In The Village, all communication between the client and server is secured using HTTPS, TLS certificates and Express enforcement to encrypt data in transit. This protects sensitive mental health and physical pain information from interception or tampering during transmission. Enforcing secure connections across all routes, APIs, and assets reinforces user trust and ensures that privacy is treated as a non-negotiable foundation rather than an optional feature.

User tracking preferences are honoured by minimising unnecessary data collection and providing clear choices around analytics or cookies. Where tracking is used to improve functionality, it is implemented transparently and only with informed consent. Respecting these preferences acknowledges user autonomy and recognises that individuals engaging with mental health and pain tracking services may have heightened concerns about digital privacy.

Users are given clear, accessible explanations about what information is collected, why it is needed, and how it is used. Privacy policies and consent prompts are written in plain language rather than legal or technical jargon. This transparency empowers users to make informed decisions and reduces uncertainty, which is particularly important in a wellbeing-focused environment.

User data is protected through secure storage practices, including encryption, access controls, and proper authentication mechanisms with JWTs and MongoDB Atlas. Sensitive information such as identity, mental health and pain information is only accessible to authorised systems and is handled according to the principle of least privilege. Regular review of security measures helps ensure that protections evolve alongside emerging risks, maintaining a safe and trustworthy platform for all users.

Web developers should be considerate of their peers

In The Village, we maintain clear inline comments and structured documentation to support maintainability and collaboration. Code is documented using meaningful function descriptions and README files within our GitHub repository, ensuring other developers can understand setup, architecture, and design decisions. Where appropriate, JSDoc-style comments are used in JavaScript to clarify parameters and return values.

To maintain code quality and consistency, we use ESLint to enforce coding standards across the MERN stack. Testing is conducted using tools such as Jest and React Testing Library to validate core functionality and reduce regressions. These tools help ensure that new changes do not unintentionally break existing features, supporting reliability and team accountability.

Version control is managed through Git and GitHub, enabling branching, commit history tracking, and collaborative development. Pull requests are used to review changes before merging into the main branch. Where possible, GitHub Actions can be configured to automatically run linting or tests on each push, supporting continuous integration and early error detection.

We recognise that the MERN stack relies heavily on open-source technologies, including React, Node.js, and MongoDB. When possible, we contribute by reporting issues, improving documentation, or following community guidelines. Professional respect is maintained through constructive communication in commits, pull requests, and discussions, fostering a positive and inclusive development environment.

Wireframes

Extra small devices - phones
Max-width: 600px

The wireframe shows a mobile app interface for 'THE VILLAGE WELLNESS APP'. At the top, there is a header with a house icon, a URL bar showing 'https://', and a plus icon. Below the header is a decorative banner with a house and plants, and the text 'THE VILLAGE WELLNESS APP'. A purple navigation bar contains a menu icon on the left and a person icon on the right. The main content area is titled 'Sign Up or Log In'. It contains three form fields: 'NAME' with the text 'Johnny Appleseed', 'EMAIL ADDRESS' with the text 'johnnyAS@email.com', and 'PASSWORD (≥)' with masked text '*****'. A purple 'SUBMIT' button is at the bottom. An arrow points from the 'Log In' link in the title to the text on the right.

SIGN UP/LOGIN PAGE

On extra small devices - there are no accompanying pictures. The form takes up the full screen to allow user-readability.

The page will be kept basic, with space used wherever possible to stop information elements from overlapping or stretching further than the screen.

Users can choose to sign up or click on the log in link to be taken to a login page.

If users click the menu button at this stage only the resources page will be available.

Users can also select the person icon to be taken to the login page

Small devices - portrait tablets and large phones
Min-width: 600px

NAVBAR
On small and extra small devices the nav bar changes from text to icons, to ensure readability and functionality. The icons allow more space between selections to prevent users from tapping on the wrong selection



USER ACCOUNT SETTINGS
At any point in time, the user can select the person icon to take them to their user settings which includes theme preferences, password update, and export data functions

MOOD JOURNAL
The mood journal functions exactly the same as the pain journal - with slightly different wording

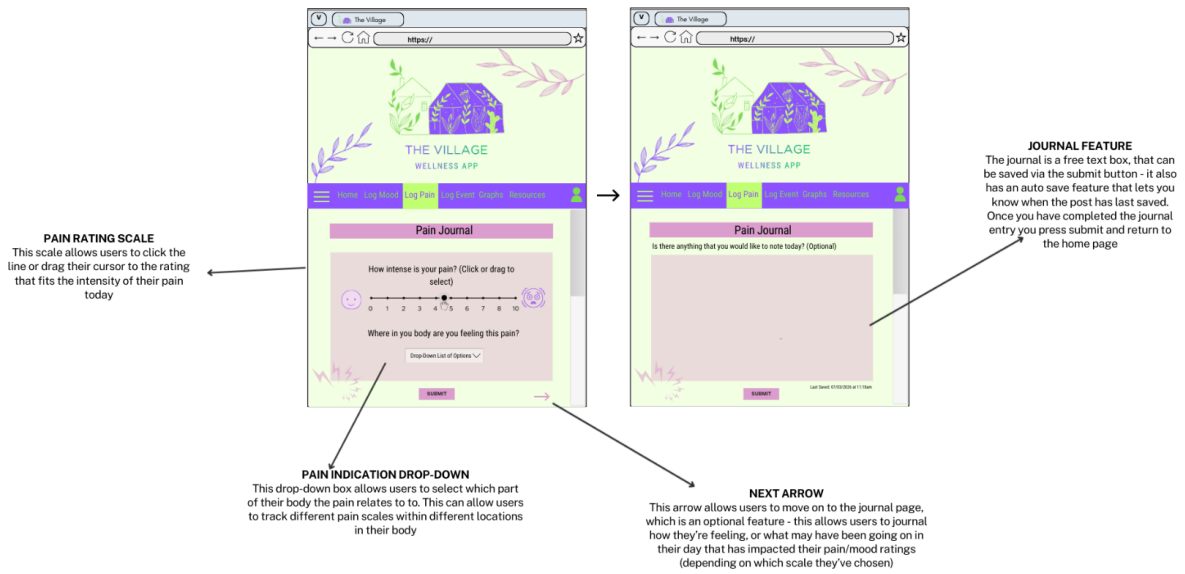
Small devices - portrait tablets and large phones
Min-width: 600px

EVENT TRACKING
The event tracking page allows users to track events in their life that may impact upon their mood or their pain. Users can add events and then in the graph page they can overlay events on top of their pain ratings or mood ratings

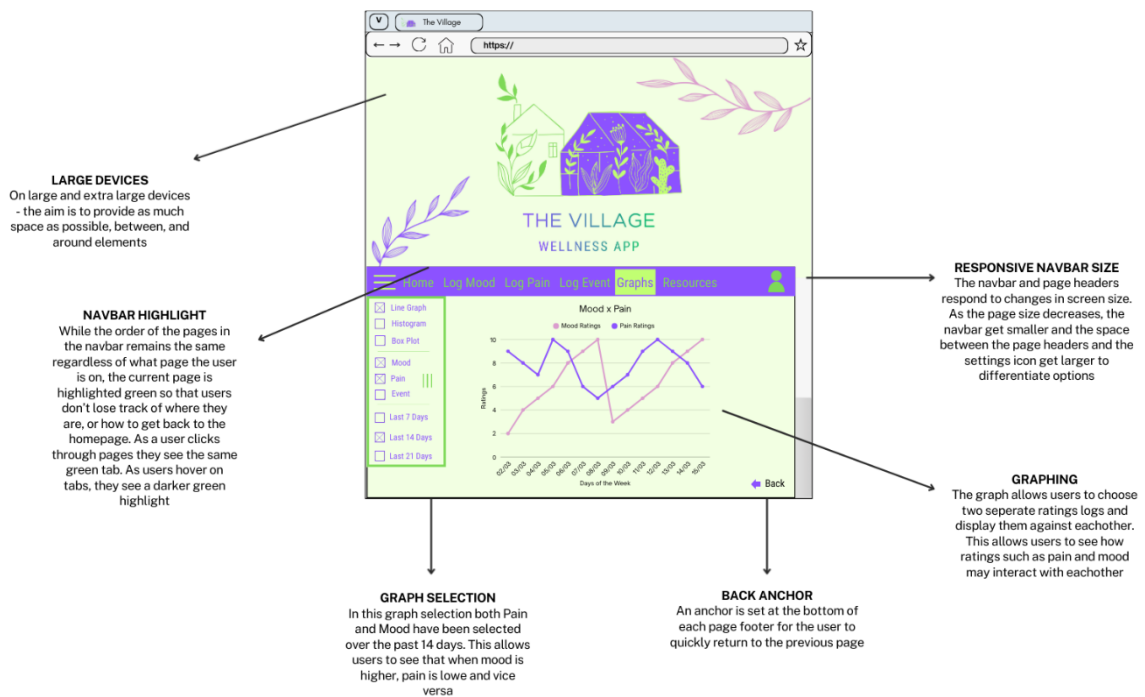


FIELDS
The fields on the events page, are similar to the mood and pain journals - however we have added a couple of fields, including date of the event as it may not have occurred on the day that the user journals

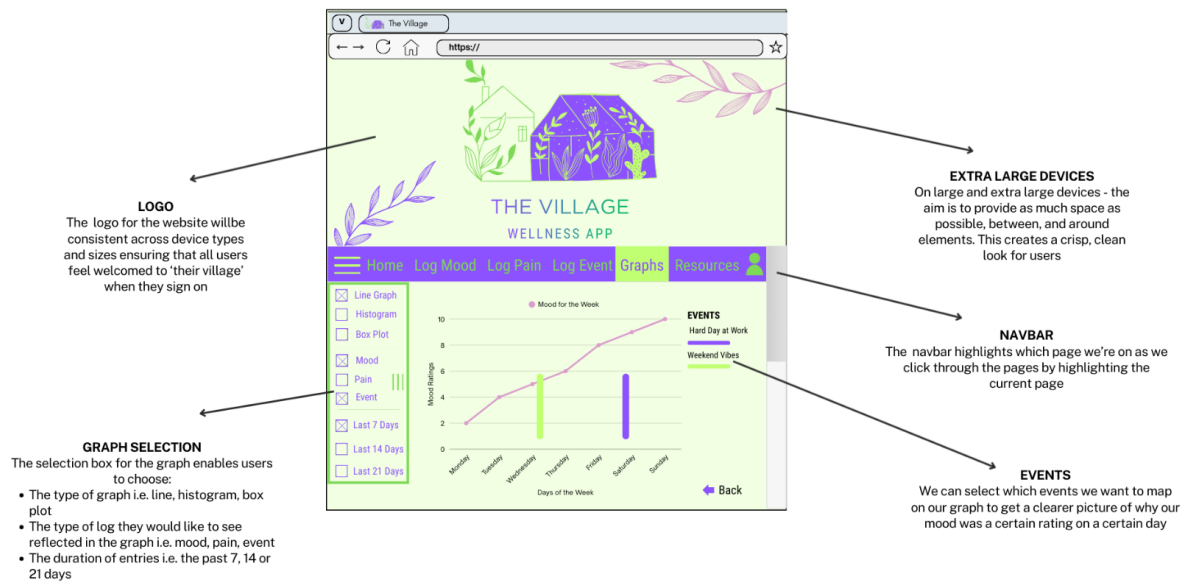
Medium devices - landscape tablets Min-width: 768px



Large devices - laptops and desktops Min-width: 992px



Extra large devices - large laptops and desktops
Min-width: 1200px



References

- **Anderson, D. and Carmichael, A. (2016).** *The Kanban Guide*. Available at: <https://kanbanguides.org> (Accessed: 7 March 2026).
- **Agile Alliance (n.d.).** "What is Agile?" Available at: <https://www.agilealliance.org/agile101/> (Accessed: 6 March 2026).
- **Atlassian (n.d.).** "Kanban vs Scrum." Available at: <https://www.atlassian.com/agile/kanban/kanban-vs-scrum> (Accessed: 7 March 2026).
- **Atlassian (n.d.).** "What is Waterfall?" Available at: <https://www.atlassian.com/agile/project-management/waterfall-methodology> (Accessed: 6 March 2026).
- **Auth0 (n.d.).** JSON Web Token introduction. Retrieved March 1, 2026, from <https://jwt.io/introduction>
- **Authgear (n.d.).** *Cryptographic failures and secure password storage*. Retrieved March 7, 2026, from <https://www.authgear.com/post/cryptographic-failures-owasp>
- **Axios (n.d.).** Axios documentation. Retrieved March 4, 2026, from <https://axios-http.com/docs/intro>
- **Beck, K. et al. (2001).** "Manifesto for Agile Software Development." Available at: <https://agilemanifesto.org> (Accessed: 6 March 2026).
- **Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P. and Stal, M. (1996).** *Pattern-Oriented Software Architecture: A System of Patterns*. Wiley.

- **Express.js (n.d.).** Express documentation. Retrieved March 4, 2026, from <https://expressjs.com/>
- **Fowler, M. (2003).** *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- **Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994).** *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- **Hohpe, G. and Woolf, B. (2004).** *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley.
- **MongoDB Documentation.** *Schema Design and Validation*. Available at: <https://www.mongodb.com/docs/manual/core/schema-validation/>
- **MongoDB Inc. (n.d.).** MongoDB documentation. Retrieved March 1, 2026, from <https://www.mongodb.com/docs/>
- **Mongoose (n.d.).** Mongoose ODM documentation. Retrieved March 1, 2026, from <https://mongoosejs.com/docs/>
- **Node Security (n.d.).** *Cryptographic recommendation summary*. Retrieved March 7, 2026, from <https://node-security.com/posts/cryptographic-recommendation-summary/>
- **Node.js Foundation (n.d.).** Node.js documentation. Retrieved March 4, 2026, from <https://nodejs.org/en/docs>
- **React Team (n.d.).** React documentation. Retrieved March 7, 2026, from <https://react.dev/learn>
- **Schwaber, K. and Sutherland, J. (2020).** “The Scrum Guide.” Available at: <https://scrumguides.org> (Accessed: 7 March 2026).
- **Singh, A. P. (2025, October 5).** Client-server architecture. AlgoMaster. Retrieved March 1, 2026 from <https://algomaster.io/learn/system-design/client-server-architecture>
- **Turner, L. (2026, January 16).** What is client-server architecture? Explained in detail. The Knowledge Academy. Retrieved March 1, 2026 from <https://www.theknowledgeacademy.com/blog/client-server-architecture/>
- **WorkOS (2026).** *Developer’s guide to strong passwords*. Retrieved March 7, 2026, from <https://workos.com/blog/developers-guide-strong-passwords>